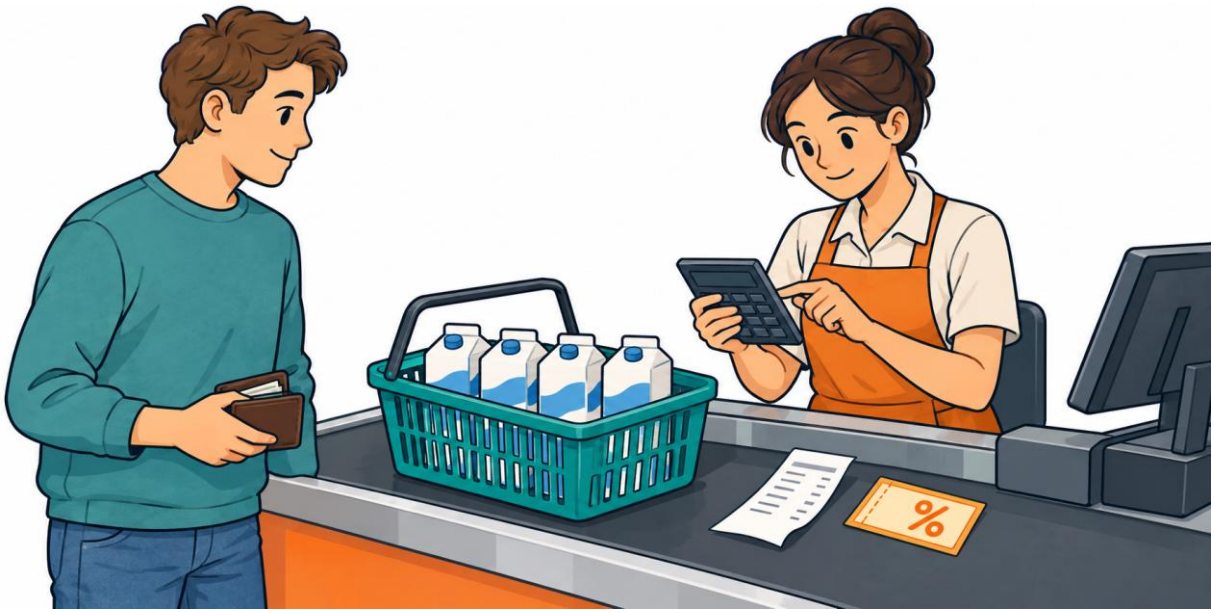
Bab 4 Operator ekspresi dan mini aplikasi

PERTEMUAN 4 • TEORI 4% • PRAKTIK 4%

Tujuan dan capaian

Menyusun ekspresi aritmetika, perbandingan, dan logika untuk menghitung diskon.

Capaian teori: CPMK032, SUB-CPMK032-01, CPMK061, SUB-CPMK061-01. Capaian praktik: CPMK032, SUB-CPMK032-02, CPMK061, SUB-CPMK061-01.



Gambar 4.1 Analogi kontekstual: operator ekspresi dan mini aplikasi

Kasir membaca jumlah barang, harga, diskon, dan ongkos. Urutan menghitung menentukan tagihan. Kalkulator tidak mengetahui apakah diskon diterapkan sebelum atau sesudah ongkos; aturan bisnis harus ditulis dengan tegas. Contoh visual ini menghubungkan operator dengan transaksi yang dapat diperiksa secara manual.

Pertanyaan pembuka

Apa hasil perbandingan? Coba jawab dengan kata-kata sendiri sebelum membaca pendalaman konsep. Hubungkan jawaban dengan objek, tindakan, dan urutan yang tampak pada gambar.

Indikator teori: Mahasiswa mampu menyusun serta mengevaluasi ekspresi Python sesuai aturan prioritas operator.

Indikator praktik: Mahasiswa mampu membentuk ekspresi aritmetika dan logika yang sesuai dengan kebutuhan kasus.

Operator aritmetika

Python menyediakan +, -, *, /, //, %, dan **. Pembagian / menghasilkan hasil bagi; // membulatkan hasil bagi ke bawah, bukan selalu menuju nol. Karena itu -7 // 3 bernilai -3. Operator % memberi sisa yang sesuai dengan pembagian lantai; 7 % 3 bernilai 1. Pangkat menggunakan **. Pembagian atau modulo dengan nol tidak sah. Satuan harus konsisten sebelum ekspresi dihitung.

Pendalaman melalui kasus

Kelompok operator memiliki tujuan berbeda: aritmetika menghasilkan besaran, perbandingan menghasilkan Boolean, dan logika menggabungkan syarat. Pisahkan subtotal dan status kelayakan diskon agar setiap nilai mudah ditelusuri. Jangan menganggap simbol yang mirip memiliki fungsi sama.

Penelusuran langkah

Tahap	Keadaan atau hasil
Harga dan jumlah	harga = 12000; jumlah = 3
Aritmetika	subtotal = 36000
Perbandingan	subtotal >= 30000 menghasilkan True
Logika	member and subtotal >= 30000 memerlukan kedua syarat

Kesalahan yang perlu dihindari

= memberi nilai; == membandingkan dua nilai. Menukarnya mengubah arti atau menghasilkan kesalahan sintaks.

Latihan konsep

Apa nilai 7 % 3 dan 7 // 3?

Jelaskan alasan jawaban Anda. Buat satu contoh masukan untuk mendukungnya dan satu perubahan masukan yang dapat menghasilkan perilaku berbeda.

Diskusi penerapan

Tunjukkan langkah yang bergantung pada keadaan sebelumnya dalam kasus di atas. Jika salah satu input diganti, tandai nilai mana yang berubah dan aturan mana yang tetap. Gunakan kontrak kasus untuk membedakan perubahan hasil yang wajar dari pelanggaran algoritma.

Prioritas dan keterbacaan

Perkalian dan pembagian dikerjakan sebelum penjumlahan. Gunakan tanda kurung untuk membuat maksud jelas: (tugas + ujian) / 2 menghitung rerata dua komponen, sedangkan tugas + ujian / 2 hanya membagi ujian. Untuk nilai berbobot, jumlah bobot harus 1 atau 100 persen sesuai representasinya. Tuliskan contoh numerik terlebih dahulu agar rumus dapat diperiksa terpisah dari kode.

Pendalaman melalui kasus

Prioritas operator mengikuti aturan bahasa. Perkalian dikerjakan sebelum penjumlahan, tetapi tanda kurung membantu menyatakan niat. Pada nilai akhir berbobot, setiap nilai dikali bobotnya sebelum dijumlahkan. Jumlah bobot harus sama dengan satu untuk rata-rata berbobot standar.

Penelusuran langkah

Tahap	Keadaan atau hasil
Tugas	80 × 0.30 = 24
Kuis	70 × 0.30 = 21
Ujian	90 × 0.40 = 36
Nilai akhir	24 + 21 + 36 = 81

Kesalahan yang perlu dihindari

Menjumlahkan semua nilai lalu mengalikan satu bobot tidak mewakili ketiga kontribusi.

Latihan konsep

Apa hasil 2 + 3 * 4 dan (2 + 3) * 4?

Jelaskan alasan jawaban Anda. Buat satu contoh masukan untuk mendukungnya dan satu perubahan masukan yang dapat menghasilkan perilaku berbeda.

Diskusi penerapan

Tunjukkan langkah yang bergantung pada keadaan sebelumnya dalam kasus di atas. Jika salah satu input diganti, tandai nilai mana yang berubah dan aturan mana yang tetap. Gunakan kontrak kasus untuk membedakan perubahan hasil yang wajar dari pelanggaran algoritma.

Perbandingan dan logika

Operator == memeriksa kesamaan, sedangkan = memberi nilai. Operator !=, <, <=, >, dan >= menghasilkan bool. and membutuhkan kedua kondisi benar; or membutuhkan setidaknya satu; not membalik nilai logis. Python melakukan short circuit, sehingga bagian kanan and tidak dievaluasi jika bagian kiri sudah salah. Ini dapat menjaga pembagian, misalnya jumlah != 0 and total / jumlah > 80.

Pendalaman melalui kasus

Batas diskon harus diterjemahkan persis. “Minimal lima puluh ribu” menggunakan >=, sedangkan “lebih dari lima puluh ribu” menggunakan >. Ambil tiga uji di sekitar ambang: 49999, 50000, dan 50001. Kasus tepat di ambang paling sering mengungkap kesalahan operator.

Penelusuran langkah

Tahap	Keadaan atau hasil
Ambang	Syarat subtotal >= 50000
Di bawah	49999: tidak mendapat diskon
Tepat batas	50000: mendapat diskon
Di atas	50001: mendapat diskon

Kesalahan yang perlu dihindari

Mengganti >= dengan > menghilangkan pelanggan yang tepat memenuhi batas minimal.

Latihan konsep

Jika member False dan subtotal 60000, apakah member and subtotal >= 50000 benar?

Jelaskan alasan jawaban Anda. Buat satu contoh masukan untuk mendukungnya dan satu perubahan masukan yang dapat menghasilkan perilaku berbeda.

Diskusi penerapan

Tunjukkan langkah yang bergantung pada keadaan sebelumnya dalam kasus di atas. Jika salah satu input diganti, tandai nilai mana yang berubah dan aturan mana yang tetap. Gunakan kontrak kasus untuk membedakan perubahan hasil yang wajar dari pelanggaran algoritma.

Aturan diskon

Kasus latihan menetapkan diskon 10 persen jika subtotal minimal Rp100000. Karena rupiah contoh berupa bilangan bulat, diskon dihitung subtotal * 10 // 100; kebijakannya membulatkan ke bawah ke rupiah utuh. Harga satuan dan jumlah barang harus positif. Aturan ini merupakan spesifikasi latihan, bukan aturan usaha universal. Batas tepat Rp100000 harus diuji karena kesalahan > dan >= mengubah kelayakan diskon.

Pendalaman melalui kasus

Tarif pengiriman per kilogram atau bagiannya memerlukan pembulatan ke atas. Untuk berat gram bulat positif, (gram + 999) // 1000 menghitung jumlah satuan tagihan. Rumus ini tidak perlu float dan mudah diuji pada 999, 1000, serta 1001 gram.

Penelusuran langkah

Tahap	Keadaan atau hasil
Berat 999	(999 + 999) // 1000 = 1
Berat 1000	1999 // 1000 = 1
Berat 1001	2000 // 1000 = 2
Biaya 1001	2 × 8000 + 2000 = 18000

Kesalahan yang perlu dihindari

gram // 1000 memberi nol kilogram untuk 999 gram dan hanya satu untuk 1001 gram, sehingga tagihan kurang.

Latihan konsep

Apa prasyarat rumus pembulatan ini?

Jelaskan alasan jawaban Anda. Buat satu contoh masukan untuk mendukungnya dan satu perubahan masukan yang dapat menghasilkan perilaku berbeda.

Diskusi penerapan

Tunjukkan langkah yang bergantung pada keadaan sebelumnya dalam kasus di atas. Jika salah satu input diganti, tandai nilai mana yang berubah dan aturan mana yang tetap. Gunakan kontrak kasus untuk membedakan perubahan hasil yang wajar dari pelanggaran algoritma.

Contoh 1 Implementasi inti operator ekspresi dan mini aplikasi

SPESIFIKASI DAN IMPLEMENTASI

subtotal = harga * jumlah layak = subtotal >= 100000 bayar = subtotal - diskon

Simpan sebagai bab04.py. Input contoh diberikan berurutan: 25000 / 4.

Kode lengkap

```
harga = int(input("Harga satuan (Rp): "))
jumlah = int(input("Jumlah barang: "))
if harga <= 0 or jumlah <= 0:
    print("Harga dan jumlah harus positif.")
else:
    subtotal = harga * jumlah
    diskon = 0
    if subtotal >= 100000:
        diskon = subtotal * 10 // 100
    bayar = subtotal - diskon
    print(f"Subtotal: Rp{subtotal}")
    print(f"Diskon: Rp{diskon}")
    print(f"Bayar: Rp{bayar}")
```

Prediksi sebelum menjalankan

diskon dimulai dari nol agar semua jalur valid memiliki nilai yang dapat digunakan.

Tulis hasil yang Anda harapkan untuk masukan contoh. Tandai baris yang membaca atau menyediakan data, baris yang mengubah keadaan, serta baris yang menghasilkan keluaran. Bandingkan prediksi dengan bukti pada bagian berikutnya setelah program dijalankan.

Pembahasan dan pengujian contoh 1

diskon dimulai dari nol agar semua jalur valid memiliki nilai yang dapat digunakan.

Empat barang seharga Rp25000 menghasilkan subtotal Rp100000, diskon Rp10000, dan bayar Rp90000.

Penolakan nilai negatif dilakukan sebelum perhitungan. Mengganti int dengan float memerlukan keputusan baru tentang pembulatan dan format uang.



Gambar 4.2 Tangkapan layar laporan eksekusi bab04.py

Kasus uji yang harus dijelaskan

Masukan / skenario	Ekspektasi
25000 x 4	Subtotal 100000; bayar 90000
99999 x 1	Diskon 0; bayar 99999
100001 x 1	Diskon 10000; bayar 90001
0 x 2	Ditolak

Jalankan setiap kasus uji dan catat hasil aktualnya. Untuk data yang ditulis langsung di dalam kode, gunakan salinan program atau panggil fungsinya dengan data uji baru. Jelaskan perbedaan hasil jika ada.

Contoh 2 Nilai akhir berbobot

SPESIFIKASI DAN IMPLEMENTASI

Nilai tugas, UTS, dan UAS masing-masing berada pada rentang 0 sampai 100. Bobot latihan 30 persen, 30 persen, dan 40 persen. Hitung nilai akhir tanpa pembulatan pada setiap komponen. Bobot ini khusus contoh perhitungan, berbeda dari bobot pertemuan pada RPS.

Simpan sebagai bab04_contoh2.py. Input contoh diberikan berurutan: 80 / 70 / 90.

Kode lengkap

```
tugas = float(input("Nilai tugas: "))
uts = float(input("Nilai UTS: "))
uas = float(input("Nilai UAS: "))
valid = (0 <= tugas <= 100 and
         0 <= uts <= 100 and 0 <= uas <= 100)
if not valid:
    print("Seluruh nilai harus 0 sampai 100.")
else:
    komponen_tugas = tugas * 0.30
    komponen_uts = uts * 0.30
    komponen_uas = uas * 0.40
    akhir = komponen_tugas + komponen_uts + komponen_uas
    print(f"Komponen: {komponen_tugas:.2f}, "
          f"{komponen_uts:.2f}, {komponen_uas:.2f}")
    print(f"Nilai akhir: {akhir:.2f}")
```

Prediksi sebelum menjalankan

Jumlah bobot 0.30 + 0.30 + 0.40 sama dengan 1. Jika seluruh nilai 100, nilai akhir harus 100.

Tulis hasil yang Anda harapkan untuk masukan contoh. Tandai baris yang membaca atau menyediakan data, baris yang mengubah keadaan, serta baris yang menghasilkan keluaran. Bandingkan prediksi dengan bukti pada bagian berikutnya setelah program dijalankan.

Pembahasan dan pengujian contoh 2

Jumlah bobot 0.30 + 0.30 + 0.40 sama dengan 1. Jika seluruh nilai 100, nilai akhir harus 100.

Data 80,70,90 memberi 24 + 21 + 36 = 81. Nilai akhir tidak boleh dihitung dengan rata-rata biasa karena bobot berbeda.

Kondisi valid menggabungkan tiga rentang dengan and. Satu nilai di luar rentang menggagalkan seluruh perhitungan.



Gambar 4.3 Tangkapan layar laporan eksekusi bab04_contoh2.py

Kasus uji yang harus dijelaskan

Masukan / skenario	Ekspektasi
80;70;90	81.00
100;100;100	100.00
0;0;0	0.00
101;80;90	Ditolak

Jalankan setiap kasus uji dan catat hasil aktualnya. Untuk data yang ditulis langsung di dalam kode, gunakan salinan program atau panggil fungsinya dengan data uji baru. Jelaskan perbedaan hasil jika ada.

Contoh 3 Biaya kirim dan pembulatan berat

SPESIFIKASI DAN IMPLEMENTASI

Setiap kilogram tertagih dikenai Rp8000. Berat dimasukkan sebagai gram bulat positif. Pecahan kilogram dibulatkan ke atas. Berat 1000 gram ditagih satu kilogram, sedangkan 1001 gram ditagih dua kilogram. Tambahkan biaya kemasan Rp2000 per paket.

Simpan sebagai bab04_contoh3.py. Input contoh diberikan berurutan: 1001.

Kode lengkap

```
berat_gram = int(input("Berat paket (gram): "))
if berat_gram <= 0:
    print("Berat harus positif.")
else:
    kg_tagihan = (berat_gram + 999) // 1000
    ongkir = kg_tagihan * 8000
    kemasan = 2000
    total = ongkir + kemasan
    print(f"Berat tertagih: {kg_tagihan} kg")
    print(f"Ongkir: Rp{ongkir}")
    print(f"Total dengan kemasan: Rp{total}")
```

Prediksi sebelum menjalankan

Untuk bilangan gram bulat positif, penambahan 999 sebelum //1000 mewujudkan pembulatan ke atas tanpa float.

Tulis hasil yang Anda harapkan untuk masukan contoh. Tandai baris yang membaca atau menyediakan data, baris yang mengubah keadaan, serta baris yang menghasilkan keluaran. Bandingkan prediksi dengan bukti pada bagian berikutnya setelah program dijalankan.

Peta pembacaan kode

Baris	Peran dalam algoritma
1	Membaca input, menetapkan, atau memperbarui nilai.
2–11	Keputusan: memilih cabang sesuai kondisi.

Pembahasan dan pengujian contoh 3

Untuk bilangan gram bulat positif, penambahan 999 sebelum //1000 mewujudkan pembulatan ke atas tanpa float.

Berat 1000 menghasilkan 1999 //1000 = 1, sedangkan 1001 menghasilkan 2000 //1000 = 2. Dua uji berdekatan ini membuktikan batas.

Kemasan ditambahkan sekali per paket. Mengalikannya dengan kilogram akan mengubah spesifikasi.



Gambar 4.4 Tangkapan layar laporan eksekusi bab04_contoh3.py

Kasus uji yang harus dijelaskan

Masukan / skenario	Ekspektasi
1 gram	1 kg; total 10000
1000 gram	1 kg; total 10000
1001 gram	2 kg; total 18000
0 gram	Ditolak

Jalankan setiap kasus uji dan catat hasil aktualnya. Untuk data yang ditulis langsung di dalam kode, gunakan salinan program atau panggil fungsinya dengan data uji baru. Jelaskan perbedaan hasil jika ada.

Praktikum kasus terbimbing

Kasus pengembangan

Buat kalkulator belanja dengan harga bulat, jumlah positif, diskon lima persen untuk subtotal minimal 50000, dan biaya kemasan 2000 setelah diskon. Tulis urutan rumus dan uji tiga titik sekitar ambang.

Latihan awal dan pengembangan

Langkah 1–4 melatih contoh inti. Langkah 5–8 diterapkan pada kasus pengembangan di atas.

1. Tuliskan rumus subtotal, diskon, dan bayar di kertas.

2. Prediksi hasil uji tepat di bawah, tepat pada, dan di atas ambang.

3. Jalankan kode dan bandingkan hasilnya dengan perhitungan manual.

4. Tambahkan biaya kirim tetap setelah diskon; jelaskan apakah ambang gratis kirim memakai subtotal atau bayar.

5. Tulis kontrak kasus laboratorium: data sah, keluaran, dan perilaku penolakan. Pisahkan aturan wajib dari pengembangan pilihan.

6. Buat dry run satu kasus normal dan satu kasus batas sebelum mengetik. Gunakan keadaan yang dapat diperiksa dengan tangan.

7. Implementasikan versi terkecil yang memenuhi kontrak, lalu tambahkan fitur satu demi satu sambil menjaga hasil lama.

8. Jalankan uji penolakan dan periksa keadaan data. Jika terjadi bug, simpan input pemicu serta hasil sebelum dan sesudah perbaikan.

Target verifikasi khusus

Kasus awal	Hasil yang perlu dibuktikan
80;70;90	81.00
100;100;100	100.00
0;0;0	0.00

Tabel di atas berasal dari contoh kedua dan menjadi latihan awal teknik pengujian. Sesuaikan data serta ekspektasinya dengan misi laboratorium; tabel ini tidak menggantikan kontrak praktikum. Kumpulkan versi final, catatan perubahan, tabel uji yang sudah terisi, dan bukti eksekusi.

Tantangan bertingkat

Kerjakan setelah praktikum. Untuk setiap tantangan, tulis spesifikasi singkat, prediksi hasil, implementasi, dan sedikitnya satu uji yang dapat membedakan solusi benar dari solusi yang tampak benar. Gunakan program contoh sebagai titik awal hanya jika Anda mampu menjelaskan tiap perubahan.

Tantangan 1. Dasar: tambahkan biaya kemasan Rp2000.

Tantangan 2. Menengah: diskon hanya untuk anggota dan subtotal minimal Rp100000.

Tantangan 3. Lanjut: buat diskon bertingkat 5 persen mulai Rp50000 dan 10 persen mulai Rp100000 tanpa menerapkan keduanya.

Tantangan 4. Pengembangan kasus: Bandingkan kebijakan diskon atas subtotal dengan diskon atas subtotal ditambah kemasan. Jelaskan mengapa kedua kebijakan menghasilkan tagihan berbeda.

Tantangan 5. Adaptasi contoh 2: Nilai tugas, UTS, dan UAS masing-masing berada pada rentang 0 sampai 100. Bobot latihan 30 persen, 30 persen, dan 40 persen. Hitung nilai akhir tanpa pembulatan pada setiap komponen. Bobot ini khusus contoh perhitungan, berbeda dari bobot pertemuan pada RPS. Ubah satu aturan secara eksplisit, lalu bandingkan versi lama dan baru dengan input yang sama.

Tantangan 6. Pembuktian contoh 3: Setiap kilogram tertagih dikenai Rp8000. Berat dimasukkan sebagai gram bulat positif. Pecahan kilogram dibulatkan ke atas. Berat 1000 gram ditagih satu kilogram, sedangkan 1001 gram ditagih dua kilogram. Tambahkan biaya kemasan Rp2000 per paket. Pilih satu uji batas pada tabel contoh, jelaskan hasilnya langkah demi langkah, dan tambahkan satu uji penolakan yang belum ditampilkan.

Kriteria penyelesaian

Tantangan 1–3: fitur benar dan uji normal serta batas tersedia. Tantangan 4: aturan baru jelas dan regresi lulus. Tantangan 5: dua kontrak serta satu input pembanding dijelaskan. Tantangan 6: tracing konsisten dan penolakan menjaga keadaan data.

Studi kasus mandiri

Kerjakan kasus berikut secara mandiri. Kumpulkan spesifikasi, pseudocode atau flowchart, kode, tabel uji, dan tangkapan layar. Jelaskan alasan keputusan algoritma serta keterbatasan solusi Anda.

Permasalahan

Buat kalkulator belanja dengan harga bulat, jumlah positif, diskon lima persen untuk subtotal minimal 50000, dan biaya kemasan 2000 setelah diskon. Tulis urutan rumus dan uji tiga titik sekitar ambang.

Pengembangan yang diminta

Bandingkan kebijakan diskon atas subtotal dengan diskon atas subtotal ditambah kemasan. Jelaskan mengapa kedua kebijakan menghasilkan tagihan berbeda.

Pertanyaan analisis

1. Apa tujuan program dan siapa penggunanya? Tuliskan masukan, keluaran, satuan, dan batas data yang sah.

2. Bagaimana urutan langkah penyelesaiannya? Nyatakan dalam pseudocode atau flowchart, termasuk jalur penolakan.

3. Data apa yang harus disimpan dan kapan nilainya berubah? Jelaskan pilihan variabel atau struktur data yang digunakan.

4. Pilih satu masukan normal dan telusuri nilai penting dari awal sampai akhir. Tunjukkan bagian yang menentukan keluaran.

5. Bagaimana perilaku program pada batas terkecil, batas terbesar, atau keadaan kosong yang relevan? Tetapkan hasil yang seharusnya sebelum menjalankan kode.

6. Apa perbedaan versi awal dan versi pengembangan? Tentukan satu kasus yang harus berubah hasilnya dan satu kasus yang harus tetap sama.

Rancangan pengujian

Kelompok uji	Tugas
Normal	Pilih masukan sah dan hitung hasil yang diharapkan.
Batas	Pilih nilai tepat pada batas aturan dan jelaskan alasannya.
Penolakan	Tentukan respons untuk masukan tidak sah dan keadaan data sesudahnya.
Regresi	Ulangi kasus versi awal setelah pengembangan.

Evaluasi rangkuman dan portofolio

Evaluasi formatif

1. Apa hasil perbandingan?

2. Mengapa perlu tanda kurung?

3. Apa peran uji tepat batas?

4. Mengapa uang contoh berupa int?

5. Pilih satu program pada bab ini. Ubah satu masukan ke nilai batas, prediksi keluaran, lalu jelaskan baris yang menentukan hasil.
6. Pilih satu penolakan atau kegagalan yang mungkin. Jelaskan apakah keadaan data berubah dan bagaimana membuktikannya.

Tugas refleksi

7. Bandingkan dua contoh program pada bab ini. Jelaskan persamaan konsep, perbedaan aturan, dan alasan perbedaan alurnya.

8. Pilih satu bagian kode yang menurut Anda paling mudah menimbulkan kesalahan. Rancang kasus uji yang dapat mengungkap kesalahan tersebut.

9. Tuliskan satu konsep yang sudah Anda pahami dan satu hal yang masih perlu dipelajari. Sertakan contoh konkret dari pekerjaan Anda.

Rangkuman konsep

- subtotal = harga * jumlah
- layak = subtotal >= 100000
- bayar = subtotal - diskon

Bukti capaian bab

Bukti	Yang harus terlihat
Rancangan	Input, proses, output, batas, dan alasan keputusan.
Implementasi	Kode berjalan dan dapat dijelaskan tanpa membaca pembahasan.
Pengujian	Kasus normal, batas, serta penolakan dengan ekspektasi mandiri.
Refleksi	Satu bug atau keputusan penting, perbaikan, dan cara verifikasi.

Sebelum melanjutkan, jelaskan hubungan bab ini dengan materi sebelumnya dan sebutkan satu keterbatasan contoh yang Anda gunakan. Catat bantuan yang diterima dan bagian yang telah diverifikasi sendiri.